

CMakeLists.txt

assets

- textures
- fonts
- music
- sfx

src

- main
- resources
- events
- app
- menu
- player
- ui
- net
- utils

Main:

creates an application, gives renderwindow, and runs game.

Resources:

Loads / unloads resources and stores paths to them and gives them IDs like string names or enums. Resources loaded in a menu's constructor and unloaded in its destructor.

Events:

Can register / deregister for SFML events and also custom game based events. Singleton managers for each type that allows you to register / deregister. Must be fed in the SFML event each window iteration of game loop.

App:

Handles the actual game loop. Delegates rendering / other logic to menu manager.

Menu:

Needs base abstract menu class with the basic functionality like rendering and updating (dt updates).

Menu manager with logic to switch between menus when an event occurs (something like event = "Switch menu" with payload of Menu::Options). Must register from the switch menu event and potentially deregister in destructor. Potentially singleton maybe not as other layers below should not interact with menu manager. Takes in the rendering window / dt updates and passes them to the currently rendered menu.

Menus themselves. Menus hold many things like widgets / game board / players / etc. menu should create these objects as needed and load resources in constructor and unload in destructor. Menu must register for needed events. Menus MUST have logic of when to switch to another menu and then emit this switch so menu manager can switch. Menus should draw their needed objects. Logic of adding / removing / moving objects as needed.

Player:

2 types of players Human and AI. Players are made to exist on something like a gameplay menu. Should have a base abstract Player class to make it easy for the menu. Players need to take in reference to some game board / other things and make their move on this board using the info / input. Should have states such as "PLAYER_TURN", "WAITING_ON_OTHER_PLAYER", etc. Player should register for a click event on the board and query the board if it's a valid move and if it is then actually make the move. AKA player alters the board's state, the board does not alter its own state.

UI:

Has a base widget class with basic functionality like rendering and updates. Has a collection of other widgets to be used as UI elements. Things like button / label / the game board (as we simplified it by letting player only alter its state) / ships themselves (can be widgets as they simply exist and are dragged and dropped) / etc.

Net:

Future multiplayer networking work. Can either use third party NAT type punching library or write own. Needs player's public IP and port etc. which can be fetched from public servers over the internet. Should have simple functions like sending and receiving a move. Should emit events when sending a move so the player / gameplay menu knows to go into a waiting state. Same on receiving a move. Have timeout logic if moves take too long or a player disconnects. No network manager is needed, just perhaps 2 classes of Network_client and Network_server as players are both. Then after moves simply interface as needed with required class.

Utils:

Logger, custom random generator for AI, math, any other utility classes.